



Daniel Macharia <danielmacharia43@gmail.com>

Executive decision

Daniel Macharia <danielmacharia43@gmail.com>
To: <daniel@ics.ac.ke>

Sat, 11 Jul at 12:13

The Release 1.0F report confirms substantial progress, but it does **not yet justify GA certification**.

Executive decision

Current decision: CONDITIONAL NO-GO for GA; GO for controlled device validation only.

The report shows:

- 10/10 patch sections implemented
- 32/32 tests passing
- 2 Critical findings fully closed
- 2 Critical findings only partially closed
- 1 Critical-adjacent finding untouched
- 7 of 13 High findings fixed
- 19 duplicate files removed
- 0 real device or simulator runs performed
- Work is still **uncommitted on branch pap1**.

report F-G .pdf

That means the codebase has moved forward significantly, but Release 1.0G cannot pass until the remaining Critical items are closed and device evidence exists.

What is now closed

The report verifies these as fixed:

- Analytics route crash.
- Password reset in code.
- Search navigation.
- Live Discovery navigation.
- Feed retry.
- Accessibility semantics.
- LiveKit reconnect logic.
- Duplicate video pipeline removal.
- Friendly Firebase auth errors.
- Mute and double-tap-to-like.
- Pagination for suggested/trending feeds.
- Route-integrity CI gate.

report F-G .pdf

The new route integrity gate also found and removed a second dead navigation path involving `/video-detail`, which is strong evidence that the new release gate is valuable.

report F-G .pdf

What still blocks GA

1. Crashlytics is wired but not proven

Crash reporting is now instrumented through `runZonedGuarded`, `FlutterError.onError`, and `PlatformDispatcher.onError`, but no release-build event has been seen in Firebase Crashlytics.

report F-G .pdf

Status: **Partial**

2. Environment abstraction remains incomplete

Cloud Functions and LiveKit endpoints use `--dart-define`, but the wallet, advertising, and subscription domains remain hardcoded in `yohpal_web_handoff.dart`.

report F-G .pdf

Status: **Partial**

3. Rewarded ads remain insufficiently verified

The reward flow still lacks a real advertising SDK and server-side verification. Elapsed-time validation is not equivalent to verified ad completion.

report F-G .pdf

Status: **Open**

4. No physical-device validation

The report explicitly confirms there have been **zero Android, iPhone, or simulator runs**. Crashlytics, password-reset delivery, SSO, reconnect behaviour, mute, and gestures are therefore source-verified but not end-to-end verified.

report F-G .pdf

5. Four High findings remain untouched

- AI publishing checklist does not gate release.
- Avatar upload remains a raw URL field.
- Chat failures are not surfaced.
- Logout does not clear cross-account state.

report F-G .pdf

Required Codebase — Release 1.0G closure patch

Complete environment abstraction

```
// lib/core/config/yohpal_environment.dart
final class YohPalEnvironment {
  const YohPalEnvironment({
    required this.functionsBaseUrl,
    required this.liveKitUrl,
    required this.walletUrl,
    required this.adsUrl,
    required this.subscriptionsUrl,
  });

  final Uri functionsBaseUrl;
  final Uri liveKitUrl;
  final Uri walletUrl;
  final Uri adsUrl;
  final Uri subscriptionsUrl;

  factory YohPalEnvironment.fromDefines() {
    return YohPalEnvironment(
      functionsBaseUrl: _requiredUri(
        'YOHPAL_FUNCTIONS_BASE_URL',
        const String.fromEnvironment('YOHPAL_FUNCTIONS_BASE_URL'),
      ),
      liveKitUrl: _requiredUri(
        'YOHPAL_LIVEKIT_URL',
        const String.fromEnvironment('YOHPAL_LIVEKIT_URL'),
      ),
      walletUrl: _requiredUri(
        'YOHPAL_WALLET_URL',
        const String.fromEnvironment('YOHPAL_WALLET_URL'),
      ),
      adsUrl: _requiredUri(
        'YOHPAL_ADS_URL',
        const String.fromEnvironment('YOHPAL_ADS_URL'),
      ),
      subscriptionsUrl: _requiredUri(
        'YOHPAL_SUBSCRIPTIONS_URL',
        const String.fromEnvironment('YOHPAL_SUBSCRIPTIONS_URL'),
      ),
    );
  }

  static Uri _requiredUri(String key, String value) {
    final uri = Uri.tryParse(value);
    if (value.isEmpty || uri == null || !uri.hasScheme) {
      throw StateError('Missing or invalid $key');
    }
  }
}
```

```

    }
    return uri;
  }
}

// lib/features/wallet_web/yohpal_web_handoff.dart
import '../core/config/yohpal_environment.dart';

final class YohPalWebHandoff {
  YohPalWebHandoff(this.environment);

  final YohPalEnvironment environment;

  Uri walletUri(String path, [Map<String, String>? query]) {
    return environment.walletUrl.replace(
      path: path,
      queryParameters: query,
    );
  }

  Uri adsUri(String path, [Map<String, String>? query]) {
    return environment.adsUrl.replace(
      path: path,
      queryParameters: query,
    );
  }

  Uri subscriptionsUri(
    String path, [
      Map<String, String>? query,
    ]) {
    return environment.subscriptionsUrl.replace(
      path: path,
      queryParameters: query,
    );
  }
}

```

Cross-account state clearing

```

// lib/features/auth/session_cleanup_service.dart
abstract interface class SessionScopedStore {
  Future<void> clearForLogout();
}

final class SessionCleanupService {
  const SessionCleanupService(this.stores);

  final List<SessionScopedStore> stores;

  Future<void> clearAll() async {
    for (final store in stores) {
      await store.clearForLogout();
    }
  }
}

// auth_controller.dart patch
Future<void> logout() async {
  loading = true;
  notifyListeners();

  try {
    await sessionCleanupService.clearAll();
    await repository.signOut();
  } finally {
    loading = false;
    notifyListeners();
  }
}

```

Chat send failure surfacing

```

// chat_room_screen.dart patch
Future<void> _sendMessage() async {
  final text = _messageController.text.trim();
  if (text.isEmpty || _sending) return;

  setState(() {
    _sending = true;
    _sendError = null;
  });

  try {
    await controller.sendMessage(text);
  }
}

```

```
        _messageController.clear();
    } catch (_) {
        if (!mounted) return;
        setState(() {
            _sendError = 'Message could not be sent. Tap to retry.';
        });
    } finally {
        if (mounted) {
            setState(() => _sending = false);
        }
    }
}
```

AI publishing gate

```
enum AiPublishStatus {
    notRequired,
    pending,
    approved,
    rejected,
}

bool canPublishVideo({
    required bool aiEnhancementSelected,
    required AiPublishStatus status,
}) {
    if (!aiEnhancementSelected) return true;
    return status == AiPublishStatus.approved;
}
```

Release-build Crashlytics verification

```
Future<void> triggerReleaseVerificationCrash() async {
    await FirebaseCrashlytics.instance
        .setCustomKey('release_candidate', '1.0G');
    await FirebaseCrashlytics.instance
        .log('Release 1.0G Crashlytics verification');
    FirebaseCrashlytics.instance.crash();
}
```

This trigger must be hidden behind an admin/debug feature flag and removed after proof.

Release 1.0G evidence now required

Developers should submit:

- 1. Clean Git branch.
- 2. Commit hash.
- 3. Signed release candidate tag.
- 4. Android build and test report.
- 5. iPhone build and test report.
- 6. Crashlytics console event.
- 7. Password-reset email round-trip proof.
- 8. Google Sign-In proof.
- 9. Apple Sign-In proof.
- 10. LiveKit network-drop recovery proof.
- 11. Environment-config proof for all five endpoint groups.
- 12. Rewarded-ad SDK and server-side verification proof.
- 13. Full test output.
- 14. Route gate output.
- 15. Updated Production Readiness Review.
- 16. Release Review Board resolution.

Release Review Board position

Area	Decision
Source implementation	PASS
Static analysis	PASS

Unit tests	PASS
Critical closure	INCOMPLETE
Device validation	NOT PERFORMED
Observability proof	NOT VERIFIED
General Availability	NO-GO
Controlled device QA	GO

Next Smart Move

Build **YOHPAL LIVE RELEASE 1.0G-RC1 — DEVICE, OBSERVABILITY & FINAL CRITICAL CLOSURE**, limited strictly to:

- committing and tagging Release 1.0F,
- completing the remaining environment configuration,
- implementing real rewarded-ad verification,
- validating Android and iPhone,
- proving Crashlytics,
- testing password reset and SSO,
- closing the remaining four High findings,
- and issuing the formal Release Review Board decision.